

Tornado – Cheat Sheet

Tornado 6.5.10 · asyncio · Python 3.9+ (6.6 upcoming: 3.11+)
versions verified 2026-10-07 · tornadoweb.org · irurueta.com

INSTALL & RUN
pip install tornado # optional: pycurl, pycares
import asyncio, tornado.web
class Hello(tornado.web.RequestHandler):
 async def get(self):
 self.write({"hello": "world"}) # dict → JSON
async def main():
 app = tornado.web.Application([
 (r"/", Hello), debug=True
])
 app.listen(8888)
 await asyncio.Event().wait() # run forever
asyncio.run(main())
debug=True = autoreload + no template/static caches +
serve traceback; or python -m tornado.autoreload app.py.
One thread, one loop per process; not WSGI; Unix targeted.

APPLICATION SETTINGS
Application(handlers, **settings)
debug autoreload serve_traceback compress_response
cookie_secret login_url xsrf_cookies xsrf_cookie_kwargs
template_path autoscape template_whitespace ui_modules
static_path static_url_prefix static_handler_class
default_handler_class/args log_function google_oauth
websocket_ping_interval/timeout/_max_message_size
6.6 upcoming: cross_origin_protection trusted_origins
HTTPServer(app, xheaders=True, ssl_options=ctx,
max_body_size=n, idle_connection_timeout=s)

TCP, IOStream & HTTPServer
class Echo(tornado.tcpserver.TCPServer):
 async def handle_stream(self, stream, address):
 while line := await stream.read_until(b"\n"):
 await stream.write(line) # StreamClosedError
Echo(ssl_options=ctx).listen(9000) # TLS
s = await TCPCient().connect("n", 9000)
socks = netutil.bind_sockets(8888, reuse_port=True)
HTTPServer(app).add_sockets(socks)

REQUESTHANDLER LIFECYCLE & I/O
initialize(**kw) → prepare() → get|post|put|patch|delete
[head|options]() → on_finish() # on_connection_close
set_default_headers() write_error()
get_argument("q", dflt) get_arguments # query + body
get_query_argument get_body_argument get_cookie
request.body.files.headers.remote_ip # JSON: prepare()
set_status(201) set_header("X-A", "b") set_cookie(...)
write(dict[str|bytes]) await flush() finish()
render("t.html", **ns) redirect("/x") send_error(404)
prepare and verb methods may be async def. Nothing can be written
after finish().

ROUTING & reverse url
(r"/books/{\d+}", BookH) # group - positional arg
(r"/books/{P\d+id+}", BookH) # named + kwargs
(r"/b/{\d+}", BookH, {'db': db}, name="book")
self.reverse_url("book", 7) # "/b/7/"
app.add_handlers(r"api.*", [(r"/v1", H)]) # by host
Application(default_handler_class=NotFound) # custom 404
RuleRouter([Rule(PathMatches("x/*"), app2),
Rule(HostMatches("a.example"), app3)])
First match wins; path args arrive as str. Anchor patterns with \$.

ERRORS, STATIC FILES & i18n
raise HTTPError(404, reason="No such book")
def write_error(self, status_code, **kw): # JSON errors
 self.finish({"code": status_code})
raise tornado.web.Finish() # stop quietly
static_path="static" + static_url("a.css") # ?v=chash>
StaticFileHandler: validate_absolute_path, get_cache_time
locale.load_translations("locale") # CSV files
locale.load_gettext_translations("locale", "app")
{{ _("%(n)d book", "%(n)d books", n) % {"n": n} }}
get_user_locale() → locale locale.format_date(dt)
No symlink escape from static_path (6.5.9)
allowed_symlink_directory, a list since 6.5.10. Serve statics from
nginx.

TEMPLATES & UI MODULES
{{ expr }} {% if x %}...{% elif %}...{% else %}...{% end %}
{% for b in books %}...{% end %} {% set v = 1 %} {% note %}
{% extends "base.html" %} {% block body %}...{% end %}
{% include "row.html" %} {% raw %} {% apply f %}
{% module Card(book) %} {% module xsrf_form_html() %}
namespace: static_url reverse_url current_user request
handler locale xsrf_token escape json_encode
class Card(tornado.web.UIModule):
 def render(self, book):
 return self.render_string("c.html", b=book)
 def embedded_css(self): return "card.css"
Application(ui_modules={"Card": Card}, ui_methods={...})
Autoscape is on; {% raw %} and autoscape=None turn it off.
JS/CSS contexts need json_encode.

SIGNED COOKIES & AUTHENTICATION
Application(cookie_secret=os.environ["COOKIE_SECRET"],
login_url="/login")
set_signed_cookie("user", name, expires_days=30,
httponly=True, secure=True, samesite="Lax")
get_signed_cookie("user", max_age_days=31) # bytes | None
cookie_secret=(0: old, 1: new), key_version=1 # rotate
def get_current_user(self): # async: set in prepare()
 v = self.get_signed_cookie("user")
 return v.decode() if v else None
@tornado.web.authenticated # none → redirect to login_url
async def get(self): ... # non-GET - 403
await loop.run_in_executor(None, hash_pw, pw) # off-loop
Signed = tamper-proof, not encrypted. Never hardcode
cookie_secret. Hash passwords with scrypt / bcrypt / argon2 off the
loop.

CHANGED SINCE OLDER TUTORIALS & BOOKS
5.0 (2018) IOLoop wraps asyncio; asyncio Future;
IOLoop.instance() deprecated
6.3 (2023) *_signed_cookie names; xsrf_cookie_name;
WSGIContainer(executor=)
6.5.9 / 6.5.10 (Sep 2026) static files no longer follow symlinks out of the
root; max_body_size in clients; query-string argument limit

XSRF & CROSS-ORIGIN PROTECTION
Application(xsrf_cookies=True) # unsafe verbs need token
<form>{% module xsrf_form_html() %} # hidden xsrf input
fetch(u, [method: "POST", headers: {"X-XSRFToken": t}])
xsrf_cookie_kwargs={"secure": True, "samesite": "Strict"}
xsrf_cookie_names="host-xsrf" # 6.3+
def check_xsrf_cookie(self): ... # exempt API routes
def check_origin(self, origin): ... # WebSocketHandler
6.6 upcoming: cross_origin_protection=True,
trusted_origins=["https://app.example"] # Sec-Fetch-Site
Bearer-token APIs are not XSRF-prone; cookie-authenticated ones are.
CORS headers: set_default_headers.

THIRD-PARTY AUTH (tornado.auth)
class G(RequestHandler, tornado.auth.GoogleOAuth2Mixin):
 async def get(self):
 if code := self.get_argument("code", None):
 tok = await self.get_authenticated_user(
 redirect_uri=CB, code=code) # google_oauth
 else:
 self.authorize_redirect(redirect_uri=CB,
 client_id=key, scope=["profile", "email"],
 response_type="code")
Check state. TwitterMixin deprecated (6.4), OpenIdMixin (6.6
upcoming). For OIDC use a library.

COROUTINES & THE EVENT LOOP
a, b = await asyncio.gather(fetch_a(), fetch_b())
r = await gen.multi({"a": fa(), "b": fb()}) # dict
await gen.with_timeout(timedelta(seconds=5), slow())
loop = IOLoop.current()
x = await loop.run_in_executor(None, blocking_fn, arg)
also: asyncio.to_thread(fn); CPU - ProcessPoolExecutor
loop.spawn_callback(job) # fire-and-forget, logged
PeriodicCallback(tick, 5000, jitter=0.1).start()
loop.add_callback(fn) # the only thread-safe call
loop.run_sync(main) # scripts
time.sleep, sync DB drivers, requests and hashing stall every
connection. An un-awaited coroutine does nothing.

QUEUES & LOCKS
q = queues.Queue(maxsize=10) # PriorityQueue, LifoQueue
await q.put(item) item = await q.get(timeout=td)
q.task_done() await q.join() async for it in q: ...
async def worker():
 async for url in q:
 try: await handle(url)
 finally: q.task_done()
tasks = [asyncio.create_task(worker()) for _ in range(5)]
async with locks.Lock(): ... locks.Semaphore(3)
ev = locks.Event(); ev.set(); await ev.wait(timeout)
cond = locks.Condition(); cond.notify_all(); await cond.wait()
Coroutine-safe, not thread-safe; timeouts raise
tornado.util.TimeoutError. Bound the queue for back-pressure.

AsyncHTTPClient
client = AsyncHTTPClient() # shared; force_instance=True
try:
 r = await client.fetch(url, method="POST",
 body=json.dumps(d), request_timeout=10,
 connect_timeout=3, follow_redirects=True,
 headers={"Content-Type": "application/json"})
except HTTPClientError as e: # 4xx / 5xx
 e.code, e.response
r = await client.fetch(rev, raise_error=False)
r.code r.body r.headers json.loads(r.body)
AsyncHTTPClient.configure(CurlAsyncHTTPClient, max_clients=50)
HTTPRequest(url, streaming_callback=f, body_producer=g)
Never call sync HTTPClient inside the loop. CurlAsyncHTTPClient
needs pycurl; fan out with gather.

WEBSOCKETS
class Chat(tornado.websocket.WebSocketHandler):
 peers = set() # per book: dict[str, set]
 def check_origin(self, origin): return origin in OK
 def open(self, book_id): self.peers.add(self)
 async def on_message(self, msg): # str | bytes
 for w in self.peers:
 try: await w.write_message(msg)
 except WebSocketClosedError: pass
 def on_close(self): ... # close(1008, "bye")
c = await websocket_connect("ws://h/ws/1") # client
await c.write_message("hi") # m = await c.read_message()
websocket_ping_interval / timeout / max_message_size
Authenticate in open; check origin; fan out across processes with Redis
pub/sub; nginx needs Upgrade headers.

STREAMING, LONG POLLING & SSE
@stream_request_body
class Upload(RequestHandler):
 def prepare(self):
 self.request.connection.set_max_body_size(1 < 30)
 def data_received(self, chunk): self.f.write(chunk)
 async def post(self): self.f.close(); self.write("ok")
 self.write(chunk); await self.flush() # stream out
 await asyncio.wait_for(cond.wait(), 30) # long poll
 self.set_header("Content-Type", "text/event-stream")
 self.write(f'[{id}]event: review[ndata: {js}\n\n']
 await self.flush() # Last-Event-ID resumes; ": ping"
 def on_connection_close(self): self.alive = False
max_body_size max_buffer_size body_timeout: nginx:
proxy_buffering off.

DATABASES & ASYNC DRIVERS
pool = await asyncpg.create_pool(dsn) # in main()
Application(handlers, pool=pool) # self.settings["pool"]
(r"/books", H, {"pool": pool}) # → initialize(self, pool)
async with self.pool.acquire() as c:
 rows = await c.fetch("select * from books")
engine = create_async_engine("postgresql+asyncpg://...")
Session = async_sessionmaker(engine, expire_on_commit=False)
async with Session() as s:
 books = (await s.execute(select(Book))).scalars()
redis.asyncio.Redis.from_url(url) AsyncMongoClient(uri)
Create pools in main(). Sync-only driver → run_in_executor. Motor
is deprecated (2025-05-14): use PyMongo AsyncMongoClient.

OPTIONS & LOGGING
from tornado.options import define, options
define("port", default=8888, type=int, help="listen port")
define("db", multiple=True, group="data")
options.parse_command_line() # --port=9000 --logging=debug
options.parse_config_file("app.conf", final=False)
loggers: tornado.access / application / general
Application(log_function=fn) enable_pretty_logging()
--logging=none # then use logging.config yourself
Containers: prefer env vars (twelve-factor). Keep secrets out of options
and logs.

TESTING
class T(tornado.testing.AsyncHTTPTestCase):
 def get_app(self): return make_app()
 def test_get(self):
 r = self.fetch("/books"); assert r.code == 200
 self.get_url("/x"); self.get_http_port()
@tornado.testing.gen_test(timeout=10)
async def test_ws(self):
 c = await websocket_connect(
 f"ws://127.0.0.1:{self.get_http_port()}/ws")
 await c.write_message("hi")
 await asyncio.Event().wait()
with ExpectLog("tornado.application", "Uncaught"): ...
sock, port = bind_unused_port() # ASYNC_TEST_TIMEOUT=10
python -m tornado.testing mod # or pytest
Fake external HTTP with a local Application. AsyncHTTPTestCase
for TLS. Legacy stop() / wait() still work.

MULTI-PROCESS DEPLOYMENT
A: same port, N copies (Linux)
app.listen(8888, reuse_port=True) # one unit per copy
B: fork after binding, before any IOLoop exists
socks = bind_sockets(8888)
tornado.process.fork_processes(0) # 0 = one per CPU
async def main():
 HTTPServer(app, xheaders=True).add_sockets(socks)
 await asyncio.Event().wait()
 asyncio.run(main())
SIGTERM - server.stop(); await close_all_connections()
Not a replacement for Unicorn / Waitress. A Tornado app cannot run
under WSGI since 6.0.

nginx IN FRONT
upstream tornado { server 127.0.0.1:8001;
server 127.0.0.1:8002; }
location / {
 proxy_pass http://tornado; proxy_http_version 1.1;
 proxy_set_header Host \$host;
 proxy_set_header X-Real-IP \$remote_addr;
 proxy_set_header X-Scheme \$scheme;
 proxy_set_header Upgrade \$http_upgrade;
 proxy_set_header Connection 'upgrade'; # WebSockets
 proxy_read_timeout 300s; proxy_buffering off; # SSE
}
location /static/ { alias /srv/bookshelf/static/; }
xheaders=True / trusted_downstream only behind a proxy you
control; TLS done at nginx.

WSGI INTEROPERABILITY
wsgi = tornado.wsgi.WSGIContainer(flask_app,
executor=ThreadPoolExecutor(8)) # 6.3+
Application([(r"/live", LiveH),
(r"/.*", fallback_handler, dict(fallback=wsgi))])
Not a replacement for Unicorn / Waitress. A Tornado app cannot run
under WSGI since 6.0.

LEGACY → CURRENT
Older tutorials / books Tornado 6.5 today
callback=_, @asynchronous async def + await (removed 6.0)
@gen.coroutine / yield / gen.Return async def / await / return
gen.engine, gen.Task, removed in 6.0: use await
YieldPoint
IOLoop.instance() / install() IOLoop.current() (deprecated 5.0)
IOLoop.current().start() asyncio.run(main()) (6.2)
get/set_secure_cookie get/set_signed_cookie (6.3)
tornado.wsgi.WSGIApplication removed in 6.0: WSGIContainer
TwitterMixin / OpenIdMixin deprecated 6.4 / 6.6: use Authlib
Motor (async MongoDB) PyMongo AsyncMongoClient
add_callback_from_signal deprecated 6.4: add_signal_handler
websocket_connect(callback=) deprecated 6.5: await

6.2 (2022) asyncio.run(main()) is the way; make_current
deprecated; client TLS verifies by default
6.5 (2025) Python 3.9+, 3.14 and free-threading; CVE-2025-47287;
stricter header / Host checks
7.0 (announced) removes CaresResolver, TwistedResolver,
add_callback_from_signal, websocket_connect(callback=)